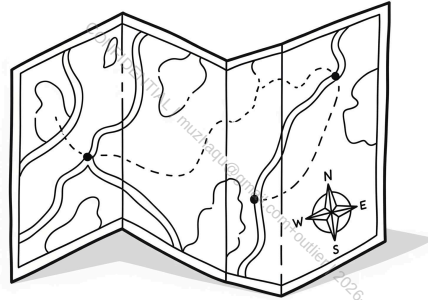




Menu Territory Code Review

Reviewer Guidelines



Project Resources

- ♦ [Reviewer channel](#)
- ♦ [Project Instructions](#)

Date	Update	Notes/Images/Links
Mar 17, 2026	→ Changed instructions to emphasize the unified diff as a primary source rather than PR links or target commits	
Mar 13, 2026	→ Added new grading dimension to include at least one "Non-Blocking Issue" rubric → Removed "missing non-critical criteria" requirement for non-blocking issue rubric → Changed Golden Review Accuracy definitions and scoring guidance	
Feb 27, 2026	Initial version	



Overview

You're a reviewer on the Code Review project. As a reviewer, you'll need to learn the following rules. Here are some things to note about this project:

1. There is no SBQ feature. If a task gets to the review layer and it's not the best, you'll need to fix it. If you need to redo everything from scratch, you can go ahead but make sure when you provide feedback, it is based on the initial attempt. This means we trust your quality to redo the task from scratch!
2. You must follow a Grading Rubric. This rubric is the cornerstone to ensuring that no bad tasks make it forward. As long as you fully understand the grading rubric, you can be sure that you are reviewing properly.
3. You must provide very good feedback. Provide proper feedback on the prompt, golden review, rubrics and the components of the rubrics, and the model failure. The more detailed and specific the feedback, the better for everyone on the project

Scoring Guidance

- In the normal scenario, we want the feedback to be structured like the following, to make it more uniform and easy to understand for CBs:
[Error Category] - Description
- If running out of time just list the error category and description of the failing error.
- You **must** score according to the rubric—no penalties based on your own judgment calls.



Reviewer Workflow

1

Review the task instructions:

<https://app.outlier.ai/en/expert/guidelines/697aafdc22e7c2497d0d1ffcai/en/expert/guidelines/697aafdc22e7c2497d0d1ffc>

2

Analyze the given repository, PR and issue title

(All of these are preprovided now rather than CB harvested)

The PR Link from the GitHub repo is not to be used for grading. The pre-provided diff is the Source of Truth, which should also match the prompt's uploaded diff (besides minor formatting differences)

Github Repo

<https://github.com/flightlessmango/MangoHud>

Repository Name and Description

MangoHud

A Vulkan and OpenGL overlay for monitoring FPS, temperatures, CPU/GPU load and more. Discord: <https://discordapp.com/invite/GJSYmBb>

Base Commit

[base_commit placeholder]

Target Commit

[target_commit placeholder]

PR Link

[pr_link placeholder]

Issue Title

[issue_title placeholder]

3

Evaluate the prompt. It should consist of three aspects:

- The Role: should clearly indicate that the model is supposed to perform a code review
- The Problem Statement: the issue that the PR is aiming to solve
 - If this is missing, then the agent is expected to infer what the goal of the PR is by looking at the repo. It has access to all of the repo exactly till the buggy target commit but does not have access to the PR discussions.
- The Diff (attached as reference text): the PR's attempted patch (*Note: This should be flawed in some way; otherwise, there would be nothing for the model to note in the review.*)
 - Due to an ENG issue, the pre-provided diff might not align with the base-to-target diff (often missing test patch changes). The pre-provided diff is the Source of Truth, so ignore the target commit if there is no alignment.

Craft your prompt

You are a senior software engineer reviewing a pull request diff in the Healthchecks repository. Healthchecks is a production cron job monitoring service built with Python 3.12, Django and a relational database (PostgreSQL, MySQL or MariaDB). Your role is to verify that the changes correctly solve the stated problem, are production ready and do not introduce regressions or subtle bugs.

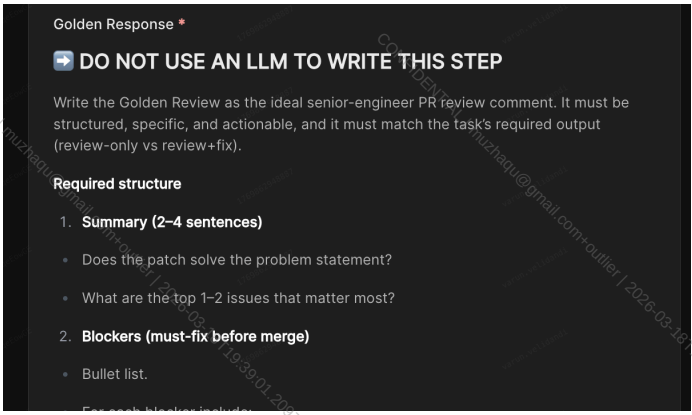
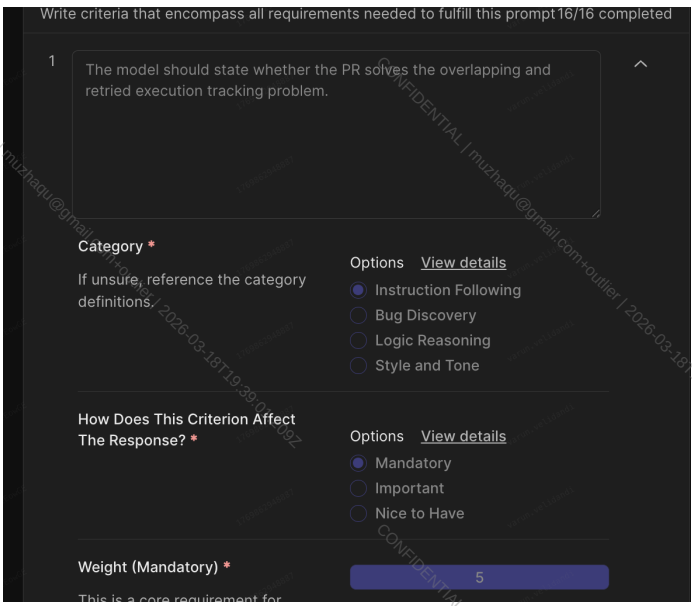
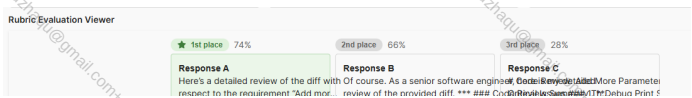
Healthchecks relies on incoming "ping" requests to track the execution of scheduled tasks. Currently, the system does not support identifying individual executions of overlapping or retried jobs, which leads to incorrect duration calculations and duplicated ping records. When clients retry failed requests, multiple identical pings can be logged, breaking idempotence and inflating execution history. Additionally, users running overlapping cron jobs cannot reliably track the start and end times of each individual run, causing the check state to become inconsistent when one run finishes while another is still active. The PR introduces support for a user-specified "run id" parameter, enabling accurate tracking of overlapping runs and laying the groundwork for making ping requests idempotent.

The commit diff is:

...

```
diff --git a/hc/api/management/commands/smtpd.py
      b/hc/api/management/commands/smtpd.py
```

```
index 21404d43..0b6326cd 100644
```

4	Evaluate the Golden Review written by the CB. It should clearly perform a thorough code review.	
5	Evaluate the Rubric	
6	Evaluate the Rubric Evaluations (The average of the 3 scores must fall below a certain threshold—this is to ensure that the task meets complexity standards.)	
7	Tally up the final score per the Grading Instructions and make any error category selections (if applicable).	

✗ No SBQs (Send Back to Queue) allowed ✗

As a reviewer, we are asking you to **fix tasks** so that they are a **4 or a 5** instead of sending them back to the attempt layer.

1. **Fix any errors you identified in the reviewer workflow, including but not limited to:**
 - a. Revision, addition, or removal of rubric criteria where necessary
 - b. Recategorization of rubric type (e.g. Bug Discovery vs. Instruction Following) where it is wrong
 - c. Updating weights based on the need of criteria and fixing the format of the rubric as needed
 - d. Fix the golden response as necessary so that it passes all the rubrics (Mandatory, Important and Nice to have) for the perfect score of 5.
2. If you think the prompt is completely bad, and you need to start from scratch, then make sure to regenerate responses and discard answers.

 Grading Rubric

Dimension	Sub- Dimension	Notes for Reviewers	1 - 2	3 - 4	5
Prompt	Category Relevance	All the prompts in this project are related to code review. Ideally, the prompt should assign a persona of a senior software engineer (or equivalent) tasked with performing a code review. Available types: - Review-only: Return a detailed PR review with issues, risks, and required changes before merge. - Review can discuss how to fix the code and still count as "Review-only" - Review + fix (Airflow-style): If issues exist, propose a fix plan and then provide a corrected patch (diff) in your response.	[Fail - Prompt Does Not Request Code Review] The prompt does not in any way indicate that the agent is expected to perform a code review. [Fail - Incorrect Type Categorization] The prompt should be categorized as review only (doesn't ask for code fixes), but it's tagged as review+fix or vice versa	N/A	The prompt asks for a code review
Prompt	Problem Statement and Diff Alignment	We need the PR (Diff) to have at least one blocking issue, so that the code review is substantive. List of issues that are considered blocking: - creating new bugs, security issues, functionality failures, correctness issues, missing core tests, edge cases, performance regression, API misuse, etc List of issues that are considered non-blocking: - poor formatting, subpar documentation, dependency	[Fail - PR Has No Blocking Issues] The PR (Diff) perfectly aligns with what the Problem Statement requests or only has non-blocking issues	N/A	There is at least one blocking issue with the PR

Dimension	Sub- Dimension	Notes for Reviewers	1 - 2	3 - 4	5
		bumps, etc If the problem statement is missing from the prompt, then the agent is expected to infer what the goal of the PR is by looking at the repo. It has access to all of the repo exactly till the buggy target commit but does not have access to the PR discussions.			
Prompt	Diff Accuracy	Make sure that the Diff Uploaded as reference text in the prompt is aligned with the pre-provided code diff. Due to an ENG issue, the pre-provided diff might not align with the base-to-target diff (often missing test patch changes). The pre-provided diff is the Source of Truth, so ignore the target commit if there is no alignment. For [Fail - Diff Misalignment] , purely formatting related differences are completely fine	[Fail - Diff Misalignment] - The diff in the prompt does not match the pre-provided diff (which is the diff between the base and target commit) [Fail - Diff Missing] - The CB did not upload a unified diff as a reference text along with the prompt	[Non-Fail - Minor Diff Formatting] - There are minor formatting differences between the diff uploaded in the prompt and the pre-provided diff, but these formatting differences do not hinder the ability to understand what changes are made.	- Presented diff aligns with what's presented perfectly
Golden Review	Review / Rubric Alignment	The Golden Review should encapsulate what an ideal response should look like—it shouldn't fail any of the Rubric's important criteria. Account for errors in rubric categorization. For example, if a criterion is incorrectly marked as nice-to-have but should have been mandatory, and the golden review fails that, then the task would fail.	[Fail - Golden Review / Rubric Misalignment] - The Golden Review fails any of the Rubric's non-optional criteria (i.e., "Important" / "Mandatory")	[Non-Fail - Golden Review / Rubric Misalignment] The Golden Review only fails the Rubric's optional criteria (i.e., "Nice-to-have")	- The Golden Review passes all of the Rubric's criteria
Golden Review	Review Accuracy	The Golden Review should be free of any objective inaccuracies, and its code changes should not introduce errors. Major/Minor Accuracy Issues: Major accuracy issues include when the statement: - Changes the technical meaning - Changes what the review is actually claiming about the code. - E.g., saying a bug is in parsing when it is actually in validation, saying a value is dropped when it is really preserved, or saying behavior is safe/unsafe when the diff shows the opposite. - Misidentifies the issue - Changes the characterization of an issue - This is for errors which misidentify what the issue is, not where it is. - Points to the wrong behavior - Attributes the problem to the wrong runtime effect or wrong codepath	[Fail - Golden Review Accuracy] - The golden review objectively contains: - 1+ major accuracy issues, - Invalid Code, - Or >3 minor accuracy issues	[Non-Fail - Golden Review Accuracy] - The golden review objectively contains 1-3 minor accuracy issues.	The Golden Review is completely accurate

Dimension	Sub-Dimension	Notes for Reviewers	1 - 2	3 - 4	5
		- Would affect grading - Could mislead someone evaluating the Golden Review against the rubric - E.g., it makes a response look correct/incorrect for the wrong reason, or supports a criterion with a fact the diff does not actually show. Anything else not listed above should be considered a minor issue.			
Golden Review	Review Format	The Golden Review should be written in the format of a Code Review from a senior programmer.	[Fail - Golden Review Format] - The Golden Review does not follow <i>any</i> reasonable Code Review format (not just the one specified by the project guidelines) - E.g., A corrected diff is provided without any context, summary, or explanation	[Non-Fail - Golden Review Format] - The Golden Review is written as a Code Review, but does not follow this format (or semantic equivalent—exact wording and order can vary): - Summary (2-4 sentences) - Blockers - Non-blocking suggestions (if applicable) - Tests / Validation - Fix Plan (if applicable) - Corrected Patch (if applicable) - The Golden Review suffers from any other formatting-related issues (e.g., improper Markdown)	- The Golden Review is written as a Code Review
Rubric Criteria	Minimum Count		[Fail - Criteria Count] - There are fewer than 15 criteria in the rubric.	N/A	- There are at least 15 criteria in the rubric
Rubric Criteria	Non-Blocking Issue	See here for definitions for what is considered non-blocking	[Fail - No Non-blocking Issues] - The rubric does not contain ANY non-blocking issues within its criteria	N/A	- The rubric contains at least one non-blocking issue in its criteria
Rubric Criteria	Overall Rubric Quality	Issues with rubric criteria are divided into three types - Major, Moderate and Minor. All the errors across rubrics are tallied at the end to arrive at a holistic rating for the entire rubric. See the next columns to understand the thresholds. See Rubric Quality Definitions section below to understand what counts as a Major, Moderate and Minor issue. Use the number of criteria that the CB wrote as	[Fail – Major Rubric Errors] - ≥ 2 criteria have major issues. [Fail – Moderate Rubric Errors] - ≥ 3 criteria have moderate or major issues. [Fail – Minor Rubric Errors] - ≥ 4 criteria have minor/moderate/major issues.	[Non-Fail – Major Rubric Errors] - 1 criteria has major issues. [Non-Fail – Moderate Rubric Errors] - 1–2 criteria have moderate/major issues and major count ≤ 1. [Non-Fail – Minor Rubric Errors] - 1–3 criteria have any issue (minor/moderate/major), and major count ≤ 1, moderate/major count ≤ 2.	- Less than or equal to 1 minor issue - No major or moderate issues

Dimension	Sub- Dimension	Notes for Reviewers	1 - 2	3 - 4	5
		the denominator while calculating % values. Numerator is the number of criteria with issues of the type (see definition below) among criteria. Take note that some issues have bespoke counting rules (mentioned under "Definition"). Do NOT double count criteria while tallying even if it has multiple issues. If a criteria has 1 major issue and 1 moderate issue, count it as having 1 major issue only.			
Rubric Evaluations	Average Below Threshold	What is the average score across the three LLM judge evaluations?	[Fail - Trivial Task] The average rubric score across the model responses is greater than or equal to 70%	N/A	The average score is less than 70%
All CB Generated Content	Unlisted Minor Errors		N/A	[Non-Fail - Unlisted Minor Errors] There are errors in the task that are not explicitly mentioned in the grading rubrics but prevent the task from being perfect.	There are no unlisted errors that you feel degrade the quality of the task

Rubric Quality Definitions

Major Issues	Missing Criteria - Critical Requirements	Definition: Count each missing rubric that ought to check whether the response points out blocking issues in the PR as one issue (see here for examples of what's blocking)
	Criteria Not Self Contained	Definition: Criterion cannot be evaluated against the model response without access to the prompt, reference text, other criteria, and/or external facts/information Every rubric must be self-contained. Imagine that you only have access to the model response and are trying to evaluate if it fulfilled the rubric item. Will you be able to evaluate accurately without referencing anything else? Some criteria are self-explanatory but often, this translates to the criteria mentioning the answer to the prompt directly. Examples of criteria that are not self-contained: - Example 1: "Response identifies the first president of the USA" - Fixed: "Response identifies the first president of the USA as George Washington" - Example 2: "The response addresses the bug mentioned in the prompt" - Fixed: "The response addresses the bug where the submit button doesn't work"

	Criteria Not Atomic - Major	Definition: Criterion groups two or more constraints that are completely unrelated, which results in a rubric item with no clear focus on what aspect of the response it's trying to evaluate. These constraints cannot be interpreted as part of a single coherent instruction but reads more like a dump of requirements. Each rubric should evaluate one thing only — no bundling of multiple behaviors. Ask yourself if the criterion is evaluating more than one idea.
	Incorrect Criteria	Definition: - Criterion checks for something that does not align with prompt requirements - Criterion contains a factual error or a misleading point - Example: "The response implements a sorting algorithm that runs in $O(n \log n)$, such as selection sort" - Criterion is not an explicit requirement in the prompt and implementing it does not make the response worse - Criterion is not at all related to the requests in the prompt NOTE: Before classifying any issue as "Incorrect criteria", see if a different, more specific error category would apply. For example, if a criterion is overly specific, you could argue that it's "incorrect", but it should still be counted as "Overfitting and Underfitting".
	Framing	Definition: Criteria that are not framed in a way such that a good response would evaluate to "Yes" or "True" Criteria should be positively <i>framed</i>. Meaning, they should all evaluate to "True" or "Yes" or "Pass" for a good response. Incorrectly framed criteria are as bad as incorrect criteria.
Moderate Issues	Overlapping or Redundant Criteria	Definition: - Criterion that is either completely redundant because other criteria completely encompass the former or multiple criteria that check for the same thing <i>partly</i> - Count each completely redundant criteria as one moderate issue or count multiple overlapping criteria as one moderate issue Redundant Scenario: Criteria 1: Response does a, b, c Criteria 2: Response does a, b Overlap Scenario: Criteria 1: Response does a, b, c Criteria 2: Response does b, c, d Note that this applies to cases where two criteria independently assess the same elements, not when a single criterion introduces and specifies related requirements. ("The response follows best code practices by ensuring that each line is under 79 characters" is acceptable.)
	Overfitting and Underfitting	Definition: Overfitting: Criteria that are overly specific, inflexible or too rigid - they incorrectly reject a subset of valid implementations Underfitting: Criteria that are overly broad, permissive or loose - they accept valid implementations, but also incorrectly accept invalid implementations too Criteria must be flexible enough to accept different valid implementations. Note that criteria can mention specific answers as long as they are provided as examples in any way. i.e. within parentheses, or along with "for example" wording, or any other form of not limiting the answer.

		Conversely, criteria must not be too permissive or overly broad, such that they would also accept invalid implementations along with valid implementations.
	Subjective Criteria	Definition: Criteria that are subjective, vague or immeasurable (e.g., “the response should have good formatting” or “code must be optimal”) A criterion should be evaluated based on whether its primary requirement is measurable, even if it includes additional context or reasoning that’s less precise Using vague or subjective qualifiers like “appropriate”, “properly”, “best practices”, “reasonable” etc without attaching explicit definitions makes criteria unmeasurable and should be flagged here. Subjectivity with certain details should be acceptable when the prompt is intentionally ambiguous and open-ended For example, for a prompt “Create an artistic website to showcase my sculptures, featuring an animated background that looks like shifting, fluid marble”; the following rubric is acceptable: “The website has a refined modern look so it could be launched as a product by a reputable company.”
	Incorrect Weights - Major	Definition: - Criteria that are objectively incorrectly weighted by two levels, i.e., 1 is selected when 5 is appropriate or vice versa Weights: 5 — Mandatory - Explicitly requested from the prompt - Core requirement for adequately answering the prompt. - Response would be unacceptable without it. - Recognition or Identification of serious blocking issues. - Possible solution or fix for new bugs/correctness issues, functionality failures, API misuse, security risks, missing core tests, performance regression, core edge cases, etc. 3 — Important - Implicitly requested from the prompt - Makes the response substantially better. - Response would be acceptable without it if all other criteria are met. - Recognition of non-blocking issues and suggestions, tests/validations. 1 — Nice to have (Optional) - Nice-to-have and necessary for a perfect response. - Response can still be strong enough without it. - Could include best code practices, libraries, language versions, formatting, style/tone etc which are not asked in the prompt explicitly. - Any response formatting or tone-related criteria.
	Criteria Not Atomic - Minor	Definition: Criterion groups two or more constraints that are only partially related. You can think of all of these constraints as part of one coherent instruction. Each rubric should evaluate one thing only — no bundling of multiple behaviors. Ask yourself if the criterion is evaluating more than one idea.

Minor Issues	Incorrect Weights - Minor	Definition: Criteria that are objectively incorrectly weighted by one level. i.e., 1v3 or 3v5 scenarios Criteria are categorized into one of 3 weight buckets: 1 (nice-to-have), 3 (important), or 5 (mandatory)
	Miscategorized Criteria	Definition: Criteria are objectively tagged with the wrong category when there is a better one available. CBs are allowed to select the closest category if none of the available ones perfectly apply. List of categories: - Instruction Following: Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements). - Bug Discovery: Verifies the model identifies and suggests solutions for bugs or issues present in the PR. - Logic Reasoning: Verifies the model identifies and suggests solutions for any reasoning flaws. - Style and Tone: Checks for any formatting or tone requirements that would make the response better looking. If the prompt asks for it explicitly, then corresponding rubrics should be tagged as “Instruction Following” rather than “Style and Tone”. This should be tagged as nice-to-have unless it’s stemming from an explicit instruction in the prompt.